

License

- **Open Source:** AGPLv3
- **Commercial:** For commercial use, a separate license is required.

For full details, please see the [LICENSE](#) file in the repository.

Go

```
go get github.com/mikhaildadaev/ulog
```

The latest stable version of `ulog` is **v1.26.12**.

Run Test

```
go test ./...
go test -bench=. ./...
go test -cover ./...
go test -race ./...
```

Key Features

- **Unified API** — One API for logs, metrics, and traces.
- **Context Extraction** — Automatic extraction `node_id` , `trace_id` , etc. from `context.Context` .
- **Colored output** — `Dark` and `Light` themes with auto-detection for TEXT format.
- **16 Field Types** — `Bool` , `Bools` , `Duration` , `Durations` , `Error` , `Errors` , `Float64` , `Floats64` , `Int` , `Ints` , `Int64` , `Ints64` , `String` , `Strings` , `Time` , `Times` .
- **SinkFile** — Non-blocking atomic rotation with `gzip` .
- **SinkHttp** — `Batching` , `Circuit Breaker` , `Deduplication` , `Retry` , `Sampling` .
- **8 Integrations** — `Discord` , `Kafka` , `Loki` , `Prometheus` , `Slack` , `Telegram` , `Tempo` , `WeChat` .

Limits

- **Async buffer:** if full, log is written synchronously *noblocking*
- **Caller information:** only for `LevelDebug` *performanceoptimization*
- **Time precision:** microseconds *6digits* — sufficient for 99% of use cases, reduces allocations
- **Deduplication cache:** in-memory only, cleared periodically *nopersistenceacrossrestarts*
- **Circuit Breaker:** resets on application restart *nopersistentstate*
- **File rotation:** checks size on each write; rotation triggered by first write exceeding limit
- **HTTP batching:** messages may be lost if application crashes before flush

- **Kafka sink:** uses REST Proxy API *notnativeKafkaprotocol* — requires Confluent REST Proxy
- **Loki sink:** uses HTTP API (`/loki/api/v1/push`) — labels must be pre-configured
- **Context extraction:** only works with values stored via `context.WithValue()`
- **Zero dependencies:** by design; no external libraries for features like Kafka native protocol

API / Core / Constructors

This page describes how to create a telemetry instance, configure all the settings, and understand each data type and field constructor.

NewTelemetry

Telemetry instance with all configuration options

```
ctx := context.Background()
ctx = context.WithValue(ctx, "node_id", "123-abc")
ctx = context.WithValue(ctx, "trace_id", "abc-123")
telemetry := ulog.NewTelemetry(
    ulog.WithExtractor("node_id", "trace_id"),
    ulog.WithFormat(ulog.FormatJson),
    ulog.WithLevel(ulog.LevelDebug),
    ulog.WithMode(ulog.ModeAsync, ulog.DefaultWriterOut, 1000),
    ulog.WithTheme(ulog.ThemeLight),
)
defer telemetry.Close()
telemetry.InfoWithContext(ctx, ulog.DataLog,
    ulog.String("message", "text"),
)
telemetry.InfoWithContext(ctx, ulog.DataMetric,
    ulog.String("name", "payments"),
    ulog.Float64("value", 99.99),
)
telemetry.InfoWithContext(ctx, ulog.DataTrace,
    ulog.String("name", "payment_processing"),
    ulog.Int64("duration", 150),
    ulog.String("span_id", "span-456"),
)
telemetry.SetExtractor()
telemetry.SetFormat(ulog.FormatText)
telemetry.SetLevel(ulog.LevelDebug)
telemetry.SetMode(ulog.ModeSync, ulog.DefaultWriterOut)
telemetry.SetTheme(ulog.ThemeDark)
telemetry.Info(ulog.DataLog,
    ulog.String("message", "text"),
)
telemetry.Info(ulog.DataMetric,
    ulog.String("name", "payments"),
    ulog.Float64("value", 99.99),
)
telemetry.Info(ulog.DataTrace,
    ulog.String("name", "payment_processing"),
    ulog.Int64("duration", 150),
    ulog.String("span_id", "span-456"),
)
```

```
{"level":"info","type":"log","message":"text","node_id":"123-abc","trace_id":"abc-123"}
{"level":"info","type":"metric","name":"payments","value":99.99,"node_id":"123-abc","trace_id":"abc-123"}
{"level":"info","type":"trace","name":"payment_processing","duration":150,"span_id":"span-456"}
```

```
[INFO] type="log" message="text"
[INFO] type="metric" name="payments" value=99.99
[INFO] type="trace" name="payment_processing" duration=150 span_id="span-456"
```

Name	Description	Values	Default
<code>WithExtractor()</code>	Auto-extract fields from <code>context.Context</code> by key names	<code>keys ...string</code>	
<code>WithFormat()</code>	Output format: structured JSON or human-readable TEXT with optional ANSI colors	<code>FormatJson</code> , <code>FormatText</code>	<code>FormatJson</code>
<code>WithLevel()</code>	Minimum log severity. Only messages at or above this level are written	<code>LevelDebug</code> , <code>LevelError</code> , <code>LevelFatal</code> , <code>LevelInfo</code> , <code>LevelWarn</code>	<code>LevelInfo</code>
<code>WithMode()</code>	Write mode: non-blocking <code>ModeAsync</code> with buffer or blocking <code>ModeSync</code>	<code>ModeAsync</code> , <code>ModeSync</code>	<code>ModeSync</code>
<code>WithTheme()</code>	ANSI color theme for TEXT output optimized for dark or light terminals	<code>ThemeDark</code> , <code>ThemeLight</code>	<code>ThemeDark</code>

Name	Description	Values
<code>TypeData</code>	Log messages, Prometheus metrics, Tempo traces	<code>DataLog</code> , <code>DataMetric</code> , <code>DataTrace</code>
<code>TypeField</code>	16 type-safe field constructors	<code>Bool</code> , <code>Bools</code> , <code>Duration</code> , <code>Durations</code> , <code>Error</code> , <code>Errors</code> , <code>Float64</code> , <code>Floats64</code> , <code>Int</code> , <code>Ints</code> , <code>Int64</code> , <code>Ints64</code> , <code>String</code> , <code>Strings</code> , <code>Time</code> , <code>Times</code>

NewTelemetryLog

Adapter for standard `log.Logger`

```
telemetry := ulog.NewTelemetry(  
    ulog.WithFormat(ulog.FormatText),  
    ulog.WithMode(ulog.ModeSync, ulog.DefaultWriterOut),  
)  
defer telemetry.Close()  
stdLogger := ulog.NewTelemetryLog(ulog.LevelError, telemetry)  
stdLogger.Print("this will be logged as ERROR")  
stdLogger.Printf("user %s failed to login", "john")  
stdLogger.Println("another error message")
```

```
[ERROR] type="log" message="this will be logged as ERROR"  
[ERROR] type="log" message="user john failed to login"  
[ERROR] type="log" message="another error message"
```

API / Core / Options

This page covers all configuration options: `Extractor` , `Format` , `Level` , `Mode` , `Theme` . Each option is shown with a working code example and expected output.

WithExtractor/SetExtractor

Automatic context extraction. Fields from `context.Context` are added to every log, metric, and trace automatically

```
ctx := context.Background()
ctx = context.WithValue(ctx, "node_id", "123-abc")
ctx = context.WithValue(ctx, "trace_id", "abc-123")
telemetry := ulog.NewTelemetry(
    ulog.WithExtractor("node_id")
)
defer telemetry.Close()
telemetry.InfoWithContext(ctx, ulog.DataLog,
    ulog.String("message", "user login"),
)
telemetry.InfoWithContext(ctx, ulog.DataMetric,
    ulog.String("name", "logins"),
    ulog.Float64("value", 1.0),
)
telemetry.InfoWithContext(ctx, ulog.DataTrace,
    ulog.String("span_id", "def"),
    ulog.String("name", "login"),
    ulog.Int64("duration", 150),
)
telemetry.SetExtractor("trace_id")
telemetry.InfoWithContext(ctx, ulog.DataLog,
    ulog.String("message", "user login"),
)
telemetry.InfoWithContext(ctx, ulog.DataMetric,
    ulog.String("name", "logins"),
    ulog.Float64("value", 1.0),
)
telemetry.InfoWithContext(ctx, ulog.DataTrace,
    ulog.String("span_id", "def"),
    ulog.String("name", "login"),
    ulog.Int64("duration", 150),
)
```

```
{
  "level":"info",
  "type":"log",
  "message":"user login",
  "node_id":"123-abc"
}
{
  "level":"info",
  "type":"metric",
  "name":"logins",
  "value":1.0,
  "node_id":"123-abc"
}
{
  "level":"info",
  "type":"trace",
  "span_id":"def",
  "name":"login",
  "duration":150,
  "node_id":"123-abc"
}
{
  "level":"info",
  "type":"log",
  "message":"user login",
  "trace_id":"abc-123"
}
{
  "level":"info",
  "type":"metric",
  "name":"logins",
  "value":1.0,
  "trace_id":"abc-123"
}
{
  "level":"info",
  "type":"trace",
  "span_id":"def",
  "name":"login",
  "duration":150,
  "trace_id":"abc-123"
}
```

WithFormat/SetFormat

Switch between Text and JSON output on the fly

```
telemetry := ulog.NewTelemetry(
    ulog.WithFormat(ulog.FormatJson),
)
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.String("message", "json message"),
)
telemetry.SetFormat(ulog.FormatText)
telemetry.Info(ulog.DataLog,
    ulog.String("message", "text message"),
)
```

```
{"level":"info","type":"log","message":"json message"}
```

```
[INFO] type="log" message="text message"
```

WithLevel/SetLevel

Filter logs by severity. Only messages at or above the configured level are written

```
telemetry := ulog.NewTelemetry(
    ulog.WithLevel(ulog.LevelDebug),
)
defer telemetry.Close()
telemetry.Debug(ulog.DataLog,
    ulog.String("message", "debug message"),
)
telemetry.Error(ulog.DataLog,
    ulog.String("message", "error message"),
)
telemetry.SetLevel(ulog.LevelInfo)
telemetry.Info(ulog.DataLog,
    ulog.String("message", "info message"),
)
telemetry.Warn(ulog.DataLog,
    ulog.String("message", "warn message"),
)
```

```
{"level":"debug","type":"log","message":"debug message"}
{"level":"error","type":"log","message":"error message"}
{"level":"info","type":"log","message":"info message"}
{"level":"warn","type":"log","message":"warn message"}
```

WithMode/SetMode

Switch between synchronous and asynchronous writing on the fly

```
telemetry := ulog.NewTelemetry(
    ulog.WithMode(ulog.ModeAsync, ulog.DefaultWriterOut, 1000),
)
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.String("message", "async message"),
)
telemetry.SetMode(ulog.ModeSync, ulog.DefaultWriterOut)
telemetry.Info(ulog.DataLog,
    ulog.String("message", "sync message"),
)
```

```
{"level":"info","type":"log","message":"async message"}
{"level":"info","type":"log","message":"sync message"}
```

WithTheme/SetTheme

Switch between Dark and Light color themes for Text output.

```
telemetry := ulog.NewTelemetry(
    ulog.WithFormat(ulog.FormatText),
    ulog.WithTheme(ulog.ThemeDark),
)
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.String("message", "dark message"),
)
telemetry.SetTheme(ulog.ThemeLight)
telemetry.Info(ulog.DataLog,
    ulog.String("message", "light message"),
)
```

```
[INFO] type="log" message="dark message"
[INFO] type="log" message="light message"
```

API / Core / Types

This page documents all data types `DataLog`, `DataMetric`, `DataTrace` and all 16 field types. Each field is shown with a working code example and expected JSON output.

Data

One API for three signal types: logs, metrics, and traces

Log

Human-readable log messages

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.String("message", "user login"),
)
```

```
{
  "level":"info",
  "type":"log",
  "message":"user login"
}
```

Metric

Machine metrics

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataMetric,
    ulog.String("name", "logins"),
    ulog.Float64("value", 1.0),
)
```

```
{
  "level":"info",
  "type":"metric",
  "name":"logins",
  "value":1.0
}
```

Trace

Distributed traces

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataTrace,
    ulog.String("span_id", "def"),
    ulog.String("name", "login"),
    ulog.Int64("duration", 150),
)
```

```
{
  "level":"info",
  "type":"trace",
  "span_id":"def",
  "name":"login",
  "duration":150
}
```

Field

16 type-safe field constructors.

Bool

Boolean field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Bool("bool", true),
)
```

```
{
  "level":"info",
  "type":"log",
  "bool":true
}
```

Bools

Boolean slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Bools("bools", []bool{true, false}),
)
```

```
{
  "level":"info",
  "type":"log",
  "bools":[true, false]
}
```

Duration

Duration field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Duration("duration", 5*time.Second),
)
```

```
{
  "level":"info",
  "type":"log",
  "duration":"5s"
}
```

Durations

Duration slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Durations("durations", []time.Duration{5*time.Second, 10*time.Second}),
)
```

```
{
  "level":"info",
  "type":"log",
  "durations":["5s","10s"]
}
```

Error

Error field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Error(fmt.Errorf("err")),
)
```

```
{
  "level":"info",
  "type":"log",
  "error":"err"
}
```

Errors

Error slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Errors([]error{fmt.Errorf("err1"), fmt.Errorf("err2")}),
)
```

```
{
  "level":"info",
  "type":"log",
  "errors":["err1","err2"]
}
```

Float64

Float64 field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Float64("float64", 3.14159),
)
```

```
{
  "level":"info",
  "type":"log",
  "float64":3.14159
}
```

Floats64

Float64 slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Floats64("floats64", []float64{1.5, 2.5}),
)
```

```
{
  "level":"info",
  "type":"log",
  "floats64":[1.5,2.5]
}
```

Int

Int field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Int("int", 42),
)
```

```
{
  "level":"info",
  "type":"log",
  "int":42
}
```

Ints

Int slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Ints("ints", []int{10, 20, 30}),
)
```

```
{
  "level":"info",
  "type":"log",
  "ints":[10,20,30]
}
```

Int64

Int64 field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Int64("int64", 1234567890),
)
```

```
{
    "level":"info",
    "type":"log",
    "int64":1234567890
}
```

Ints64

Int64 slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Ints64("ints64", []int64{1234567890, 9876543210}),
)
```

```
{
    "level":"info",
    "type":"log",
    "ints64":[1234567890,9876543210]
}
```

String

String field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.String("string", "str"),
)
```

```
{
  "level":"info",
  "type":"log",
  "string":"str"
}
```

Strings

String slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Strings("strings", []string{"str1", "str2", "str3"})
)
```

```
{
  "level":"info",
  "type":"log",
  "strings":["str1","str2","str3"]
}
```

Time

Time field

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
  ulog.Time("time", time.Date(2026, 4, 22, 12, 0, 0, 0, time.UTC)),
)
```

```
{
  "level":"info",
  "type":"log",
  "time":"2026-04-22T12:00:00.000000+00:00"
}
```

Times

Time slice

```
telemetry := ulog.NewTelemetry()
defer telemetry.Close()
telemetry.Info(ulog.DataLog,
    ulog.Times("times", []time.Time{time.Date(2026, 4, 22, 12, 0, 0, 0, time.UTC), time.Date(2025, 4, 22, 12, 0, 0, 0, time.UTC)})
)
```

```
{
    "level": "info",
    "type": "log",
    "times": ["2026-04-22T12:00:00.000000+00:00", "2025-04-22T12:00:00.000000+00:00"]
}
```

API / SinkFile / Constructors

This page documents `SinkFile`, which provides non-blocking atomic file rotation with `gzip` compression. Your service never blocks during log rotation or compression.

NewSinkFile

Atomic file rotation with `gzip` compression. Non-blocking — your service won't stall during rotation

```
var writer io.Writer = ulog.DefaultWriterOut
sinkFile, err := ulog.NewSinkFile("app.log",
    ulog.WithFileMaxAge(30),
    ulog.WithFileMaxBackups(10),
    ulog.WithFileMaxSize(100),
)
if err != nil {
    fmt.Fprintf(ulog.DefaultWriterErr, "ulog: %v - using stdout instead\n", err)
} else {
    defer sinkFile.Close()
    writer = sinkFile
}
telemetry := ulog.NewTelemetry(
    ulog.WithFormat(ulog.FormatJson),
    ulog.WithMode(ulog.ModeAsync, writer, 10000),
)
defer telemetry.Close()
telemetry.Error(ulog.DataLog,
    ulog.String("message", "critical error"),
    ulog.String("service", "billing"),
)
```

Name	Description	Default
<code>WithFileMaxAge(dayCount)</code>	Maximum days to keep old log files	30
<code>WithFileMaxBackups(fileCount)</code>	Maximum number of old log files to keep	10
<code>WithFileMaxSize(fileSize)</code>	Maximum file size <i>MB</i> before rotation	100

API / SinkFile / Params

This page covers all configuration options for `SinkFile` : maximum file age, backup count, and file size before rotation. Each option is shown with a working code example and expected behavior.

WithFileMaxAge

Sets the maximum number of days to keep old log files. Files older than this will be automatically deleted during rotation.

```
sinkFile, err := ulog.NewSinkFile("app.log",
    ulog.WithFileMaxAge(30),
)
if err != nil {
    fmt.Fprintf(ulog.DefaultWriterErr, "ulog: %v\n", err)
}
defer sinkFile.Close()
```

WithFileMaxBackups

Sets the maximum number of old log files to keep. When this limit is exceeded, the oldest backup files are deleted during rotation.

```
sinkFile, err := ulog.NewSinkFile("app.log",
    ulog.WithFileMaxBackups(10),
)
if err != nil {
    fmt.Fprintf(ulog.DefaultWriterErr, "ulog: %v\n", err)
}
defer sinkFile.Close()
```

WithFileMaxSize

Sets the maximum file size in megabytes before rotation is triggered. When the current log file exceeds this size, it is renamed, compressed, and a new file is started.

```
sinkFile, err := ulog.NewSinkFile("app.log",
    ulog.WithFileMaxSize(100),
)
if err != nil {
    fmt.Fprintf(ulog.DefaultWriterErr, "ulog: %v\n", err)
}
defer sinkFile.Close()
```

API / SinkHTTP / Constructors

This page documents `SinkHttp`, a production-ready HTTP sink with batching, circuit breaker, deduplication, retry, and sampling. Your service never blocks during network delivery.

NewSinkHttp

Creates an HTTP sink for sending logs to a remote endpoint with `Batching`, `Circuit Breaker`, `Deduplication`, `Retry`, `Sampling` built in

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpBatch(100, 5*time.Second),
    ulog.WithHttpCircuitBreaker(10, 10*time.Second),
    ulog.WithHttpDedupWindow(5*time.Second),
    ulog.WithHTTPHeader("Authorization", "Bearer token"),
    ulog.WithHttpRetry(3, time.Second),
    ulog.WithHttpSampleRate(100),
    ulog.WithHttpTimeout(30*time.Second),
)
defer sinkHttp.Close()
telemetry := ulog.NewTelemetry(
    ulog.WithFormat(ulog.FormatJson),
    ulog.WithMode(ulog.ModeAsync, sinkHttp, 10000),
)
defer telemetry.Close()
telemetry.Error(ulog.DataLog,
    ulog.String("message", "payment failed"),
    ulog.String("service", "billing"),
)
```

Name	Description
<code>WithHttpBatch(size, flushInterval)</code>	Batch messages: send up to <code>size</code> messages or every <code>flushInterval</code>
<code>WithHttpCircuitBreaker(maxFailures, timeout)</code>	Open circuit after <code>maxFailures</code> errors, wait <code>timeout</code> before recovery
<code>WithHttpDedupWindow(window)</code>	Ignore duplicate messages within <code>window</code> time
<code>WithHttpDisabledBatch()</code>	Disable message batching <i>sendimmediately</i>
<code>WithHttpDisabledCircuit()</code>	Disable Circuit Breaker
<code>WithHttpDisableKeepAlive()</code>	Disable HTTP Keep-Alive connections
<code>WithHttpFilterData(type)</code>	Filter by data type: <code>DataLog</code> , <code>DataMetric</code> , <code>DataTrace</code>
<code>WithHttpFilterLevel(level)</code>	Filter by minimum level: <code>LevelDebug</code> , <code>LevelError</code> , <code>LevelFatal</code> , <code>LevelInfo</code> , <code>LevelWarn</code>
<code>WithHttpFormatter(fn)</code>	Custom formatter function <code>func(attributes, fields) ([]byte, error)</code>
<code>WithHTTPHeader(key, value)</code>	Add custom HTTP header
<code>WithHttpMethod(method)</code>	HTTP method: <code>POST</code> , <code>PUT</code> , etc.
<code>WithHttpRetry(maxRetries, backoff)</code>	Retry failed requests up to <code>maxRetries</code> times with exponential <code>backoff</code>
<code>WithHttpSampleRate(rate)</code>	Sample 1 out of <code>rate</code> messages for non-error levels
<code>WithHttpSampleWindow(window)</code>	Reset sample counter every <code>window</code>
<code>WithHttpTimeout(timeout)</code>	HTTP client timeout

API / SinkHTTP / Factories

This page documents ready-to-use factories for `Discord` , `Kafka` , `Loki` , `Prometheus` , `Slack` , `Telegram` , `Tempo` , `WeChat` . Each factory is a pre-configured `SinkHttp` with the right formatter, headers, and filters.

NewSinkDiscord

Sends error logs to a Discord channel via webhook.

```
sinkDiscord := ulog.NewSinkDiscord("https://discord.com/api/webhooks/...", "MyBot", "")
defer sinkDiscord.Close()
```

NewSinkKafka

Sends logs to Apache Kafka via REST Proxy for stream processing.

```
sinkKafka := ulog.NewSinkKafka("http://kafka-rest:8082", "logs")
defer sinkKafka.Close()
```

NewSinkLoki

Sends logs to Grafana Loki for storage and querying.

```
sinkLoki := ulog.NewSinkLoki("http://loki:3100",map[string]string{"app": "myapp", "env": "pro"},
defer sinkLoki.Close()
```

NewSinkPrometheus

Sends metrics to Prometheus in exposition format.

```
sinkPrometheus := ulog.NewSinkPrometheus("http://pushgateway:9091")
defer sinkPrometheus.Close()
```

NewSinkSlack

Sends error logs to a Slack channel via webhook.

```
sinkSlack := ulog.NewSinkSlack("https://hooks.slack.com/services/...", "ULog", ":robot:", "",
defer sinkSlack.Close()
```

NewSinkTelegram

Sends error logs to a Telegram chat via bot API.

```
sinkTelegram := ulog.NewSinkTelegram("botToken", "chatID")
defer sinkTelegram.Close()
```

NewSinkTempo

Sends traces to Grafana Tempo for distributed tracing.

```
sinkTempo := ulog.NewSinkTempo("http://tempo:4318")
defer sinkTempo.Close()
```

NewSinkWechat

Sends error logs to WeChat Work via webhook.

```
sinkWechat := ulog.NewSinkWechat("https://qyapi.weixin.qq.com/cgi-bin/webhook/send?key=...")
defer sinkWechat.Close()
```

API / SinkHTTP / Params

This page covers all configuration options for `SinkHttp` : batching, circuit breaker, deduplication, retry, sampling, and more. Each option is shown with its default value and a brief description.

WithHttpBatch

Batch messages: send up to `size` messages or every `flushInterval` .

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpBatch(100, 5*time.Second),
)
defer sinkHttp.Close()
```

WithHttpCircuitBreaker

Open circuit after `maxFailures` errors, wait `timeout` before recovery.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpCircuitBreaker(10, 10*time.Second),
)
defer sinkHttp.Close()
```

WithHttpDedupWindow

Ignore duplicate messages within `window` time.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpDedupWindow(5*time.Second),
)
defer sinkHttp.Close()
```

WithHttpDisabledBatch

Disable message batching *sendimmediately*.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpDisabledBatch(),
)
defer sinkHttp.Close()
```

WithHttpDisabledCircuit

Disable Circuit Breaker.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpDisabledCircuit(),
)
defer sinkHttp.Close()
```

WithHttpDisableKeepAlive

Disable HTTP Keep-Alive connections.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpDisableKeepAlive(),
)
defer sinkHttp.Close()
```

WithHttpFilterData

Filter by data type: `DataLog` , `DataMetric` , `DataTrace` .

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpFilterData(ulog.DataLog),
)
defer sinkHttp.Close()
```

WithHttpFilterLevel

Filter by minimum level: `LevelDebug` , `LevelError` , `LevelFatal` , `LevelInfo` , `LevelWarn` .

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpFilterLevel(ulog.LevelError),
)
defer sinkHttp.Close()
```

WithHttpFormatter

Custom formatter function `func(attributes, fields) ([]byte, error)`.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpFormatter(func(attributes writeAttributes, fields []Field) ([]byte, error) {
        // Custom formatting logic
        return json.Marshal(fields)
    }),
)
defer sinkHttp.Close()
```

WithHTTPHeader

Add custom HTTP header.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHTTPHeader("Authorization", "Bearer token"),
    ulog.WithHTTPHeader("X-Custom", "value"),
)
defer sinkHttp.Close()
```

WithHttpMethod

HTTP method: `POST`, `PUT`, etc.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpMethod("POST"),
)
defer sinkHttp.Close()
```

WithHttpRetry

Retry failed requests up to `maxRetries` times with exponential `backoff`.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpRetry(3, time.Second),
)
defer sinkHttp.Close()
```

WithHttpSampleRate

Sample 1 out of `rate` messages for non-error levels.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpSampleRate(100),
)
defer sinkHttp.Close()
```

WithHttpSampleWindow

Reset sample counter every `window` .

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpSampleWindow(1*time.Minute),
)
defer sinkHttp.Close()
```

WithHttpTimeout

HTTP client timeout.

```
sinkHttp := ulog.NewSinkHttp("http://localhost:8080/logs",
    ulog.WithHttpTimeout(30*time.Second),
)
defer sinkHttp.Close()
```